Eksplorasi DBMS NoSQL: InfluxDB

Laporan Project 2

Disusun untuk memenuhi mata kuliah IF4040 Pemodelan Data Lanjut

oleh:

Nayla Zahira 13523079



PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

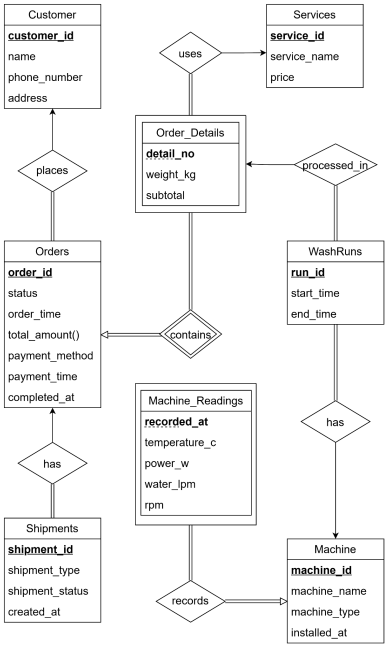
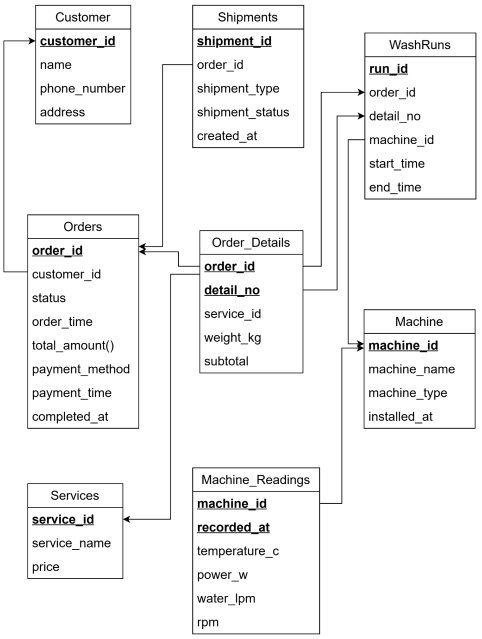
1. Deskripsi Umum

Studi kasus yang dipilih merupakan sistem pengelolaan layanan laundry kiloan. Sistem ini mencakup proses pemesanan, pemrosesan cucian, dan pengiriman. Setiap pelanggan dapat membuat satu atau lebih pesanan dan setiap pesanan terdiri atas satu atau lebih detail pesanan. Setiap detail pesanan akan mencatat layanan yang dipilih, berat cucian, dan subtotal biaya. Sistem menyediakan tiga jenis layanan, yaitu *regular wash & fold*, *express wash & fold*, serta *bedcover & linen*. Setiap pesanan harus dibayar di awal menggunakan metode tunai, transfer bank, atau QRIS. Setelah pembayaran berhasil, pesanan diproses sesuai alur operasional, mulai dari penjemputan jika diperlukan, proses pencucian, pengiriman jika diperlukan, hingga pesanan selesai.

Dalam sistem ini, setiap detail pesanan diproses melalui satu atau lebih sesi mesin *(wash run)*. Umumnya terdapat 2 tahap yaitu sesi pencucian dan sesi pengeringan. Setiap mesin mengirimkan data sensor secara berkala setiap lima menit, yang meliputi suhu, daya listrik, debit air, dan kecepatan putaran. Data sensor tersebut dikaitkan dengan sesi pencucian berdasarkan rentang waktu berlangsungnya sesi, sehingga sistem dapat menghitung konsumsi energi pada setiap sesi serta menganalisis konsumsi energi berdasarkan jenis layanan laundry.

2. Model Data Relasional

Berdasarkan deskripsi sistem, dilakukan pemodelan dalam 2 diagram sebagai berikut:

Entity Relationship Diagram	Relational Diagram
	

- Customers: data pengguna yang terdiri dari nama, nomor, dan alamat.
- Orders: data transaksi pesanan meliputi status pesanan, waktu pesan, waktu pesanan selesai, total harga pesanan, metode pembayaran, dan waktu pembayaran berhasil.
- Order_Details: rincian kuantitas per layanan dalam satu pesanan, meliputi berat dan subtotal. Order_Details merupakan *weak entity* karena identitasnya bergantung pada Orders dan menggunakan detail_no sebagai *discriminator*.
- Services: informasi layanan yang tersedia. Setiap entri meliputi nama dan harga.

- Shipments: informasi pengiriman, baik berupa *pickup* ataupun *delivery*. Setiap entri meliputi data jenis pengiriman, status, dan waktu pembuatan pengiriman.
- Machine: informasi mesin yang tersedia pada sistem. Setiap entri mesin meliputi nama, tipe, dan waktu instalasi.
- WashRuns: informasi sesi berjalannya mesin. Setiap entri meliputi waktu mulai dan waktu selesai.
- Machine_Readings: data rekaman berkala dari sensor mesin yang meliputi data suhu, daya listrik, debit air, dan kecepatan putaran. Machine_Readings ini merupakan *weak entity* karena identitasnya bergantung pada Machine dan menggunakan recorded_at (waktu data direkam) sebagai *discriminator*.

Tabel 2.1 Penjelasan Relasi

Relasi	Partisipasi	Keterangan
customer - *places* - order (1:N)	customer *partial*, order *total*	Satu pelanggan dapat memiliki satu atau lebih pesanan. Setiap pesanan harus dimiliki oleh tepat satu pelanggan.
order - *contains* - order_detail (1:N)	order *total*, order_detail *total*	Satu pesanan terdiri atas satu atau lebih detail pesanan. Setiap detail pesanan harus menjadi bagian dari tepat satu pesanan.
order_detail - *uses* - service (N:1)	order_detail *total*, service *partial*	Setiap detail pesanan menggunakan tepat satu layanan. Satu layanan dapat digunakan pada nol atau lebih detail pesanan.
order - *has* - shipment (1:N)	order *partial*, shipment *total*	Satu pesanan dapat memiliki nol atau lebih data pengiriman. Setiap data pengiriman harus terkait dengan tepat satu pesanan.
order_detail - *processed_in* - wash_run (1:N)	order_detail *partial*, wash_runs *total*	Satu detail pesanan dapat diproses melalui nol atau lebih sesi mesin. Setiap sesi mesin harus terkait dengan tepat satu detail pesanan.
machine - *executes* - wash_run (1:N)	machine *partial,* wash_runs *total*	Satu mesin dapat menjalankan nol atau lebih sesi mesin. Setiap sesi mesin harus dijalankan oleh tepat satu mesin.
machine - *records* - machine_reading (1:N)	machine *total*, machine_reading *total*	Satu mesin memiliki satu atau lebih rekaman data sensor. Setiap rekaman data sensor harus berasal dari tepat satu mesin.

Tabel 2.2 Penjelasan Constraints

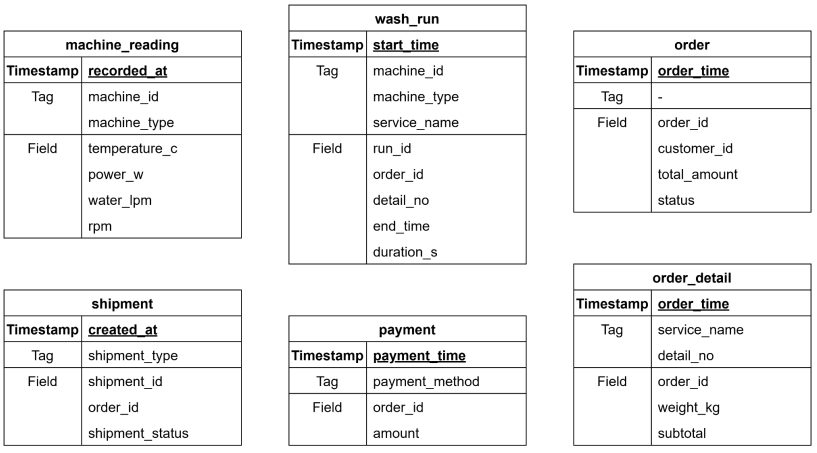
Table	Columns and constraints
customers	customer_id PK; name (not null), phone_number (not null), address (not null)
services	service_id PK; service_name; price (rupiah per kg, > 0)
orders	order_id PK; customer_id FK; status in {waiting_payment, pickup, processing, delivery, completed}; order_time; completed_at

	(nullable); total_amount (> 0); payment_method in {cash, bank_transfer, qris} (nullable); payment_time (nullable);
order_details	(order_id, detail_no) PK; detail_no > 0; service_id FK; weight_kg (not null) (>0); subtotal (> 0)
shipments	shipment_id PK; order_id FK; shipment_type in {pickup, delivery}; shipment_status in {pending, in_progress, completed}; created_at; UNIQUE (order_id, shipment_type);
machines	machine_id PK; machine_name; machine_type in {washer, dryer}; installed_at (date)
wash_runs	run_id PK; machine_id FK ; (order_id, detail_no) FK; start_time (not null), end_time (not null); CHECK (end_time > start_time)

3. Model Data NoSQL: Time-series

Pada InfluxDB, data dimodelkan menggunakan pendekatan *time-series*, yaitu setiap *point* memiliki *timestamp* yang menunjukkan waktu terjadinya suatu kejadian. Data dikelompokkan ke dalam *measurement* berdasarkan jenis informasi yang disimpan. Setiap *point* dapat memiliki *tag* sebagai atribut untuk identifikasi dan pengelompokan, serta *field* sebagai tempat menyimpan nilai yang diukur atau atribut lainnya.

Pada sistem ini, model data InfluxDB dibagi menjadi enam *measurement* sebagai berikut.



Tabel 3.1 Penjelasan Measurement

Measurement	Timestamp	Deskripsi
machine_reading	recorded_at	Menyimpan pembacaan sensor mesin, meliputi suhu, daya listrik, debit air, dan kecepatan putaran. Tag machine_id dan machine_type digunakan untuk membedakan mesin.
wash_run	start_time	Menyimpan sesi penggunaan mesin. Waktu selesai disimpan pada field end_time dalam utc satuan detik, beserta durasi sesi. Tag service_name menunjukkan jenis layanan yang terkait dengan sesi tersebut.

order	order_time	Menyimpan kejadian pembuatan pesanan, termasuk identitas pesanan, total harga, dan status terkini.
order_detail	order_time	Menyimpan rincian layanan dalam pesanan, termasuk berat cucian dan subtotal. Tag detail_no membedakan rincian dalam pesanan yang sama.
payment	payment_time	Menyimpan kejadian pembayaran untuk pesanan yang telah dibayar.
shipment	created_at	Menyimpan kejadian pembuatan pengiriman, baik penjemputan maupun pengantaran, beserta statusnya.

Tabel 3.2 Aturan Perancangan

Aturan	***Penjelasan***
Pemilihan timestamp	Setiap measurement menggunakan atribut waktu kejadian yang direpresentasikan. Timestamp harus unik dalam kombinasi measurement dan tag set.
Pemilihan tag	Tag digunakan untuk filter atau pengelompokan atribut dengan kardinalitas rendah. Pada sistem ini: machine_id, machine_type, service_name, detail_no, payment_method, dan shipment_type.
ID disimpan sebagai field	order_id, customer_id, run_id, dan shipment_id disimpan sebagai field untuk menghindari penambahan series akibat ID yang memiliki banyak nilai unik. Konsekuensinya, pencarian dan join berdasarkan ID butuh pemrosesan tambahan.
Status sebagai field	status dan shipment_status disimpan sebagai field. Perubahan status menulis ulang point yang memiliki measurement, tag set, dan timestamp yang sama. Dengan demikian, point hanya menyimpan status terkini, bukan riwayat.
Denormalisasi data master	customers, services, dan machines tidak dijadikan measurement karena data bersifat statis. Atribut yang dibutuhkan (machine_type, service_name) disalin ke point sebagai tag agar query tidak perlu join.
Relasi berbasis waktu	machine_reading tidak menyimpan referensi langsung ke sesi mesin. Pembacaan dikaitkan dengan wash_run melalui rentang waktu sesi, yaitu start_time ≤ recorded_at < end_time.
Penanganan integritas data	Tidak ada foreign key, CHECK constraint, atau transaksi. Konsistensi antardata dijaga oleh proses pemuatan dan aplikasi.

4. Query

4.1. Query DML

1. Menambahkan pesanan beserta detailnya
2. Mencatat pembayaran dan memperbarui status pesanan menjadi *pickup*

3. Menambahkan sekumpulan data sensor mesin (192 pembacaan, 16 mesin × 1 jam)
4. Mengoreksi nilai satu pembacaan sensor mesin
5. Menghapus data sensor mesin yang lebih lama dari tanggal tertentu

Implementasi: https://github.com/naylzhra/Project2_13523079/tree/main/writes

4.2. Query Read

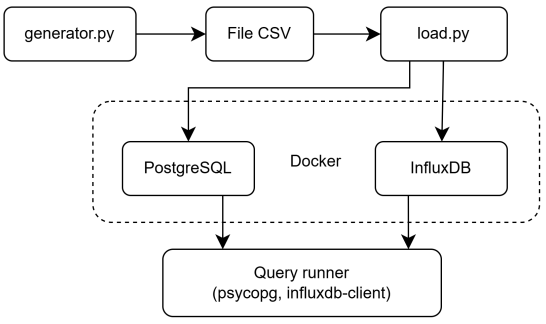
1. Jumlah pesanan per hari
2. Jumlah total pembayaran per hari
3. Jumlah total berat cucian per hari
4. Pendapatan per jenis layanan laundry (hanya pesanan yang sudah dibayar)
5. Jumlah pesanan berdasarkan status
6. Rata-rata suhu dan daya per mesin per jam
7. Total energi (kWh) per jenis layanan laundry

Implementasi: https://github.com/naylzhra/Project2_13523079/tree/main/queries

5. Arsitektur Sistem

Sistem terdiri atas dua basis data, yaitu PostgreSQL 18 dan InfluxDB 2.7.12, yang masing-masing dijalankan di container Docker terpisah dan didefinisikan dalam satu file compose.yaml. Keduanya dijalankan di lingkungan Docker yang sama sehingga memiliki jalur jaringan dan overhead virtualisasi yang setara selama pengujian. Saat pertama kali dijalankan, PostgreSQL membuat skema otomatis dari skrip di folder sql, sedangkan InfluxDB membuat organisasi laundry dan bucket laundry_metrics dengan retensi tidak terbatas.

Pada awal *setup,* data dibuat oleh *seed generator* (generator.py) dengan seed 42. Data yang dibuat mensimulasikan operasional laundry selama 30 hari. Data ini kemudian disimpan dalam bentuk file CSV di folder data. Selanjutnya, loader (load.py) akan membaca file CSV tersebut dan memuatnya ke kedua basis data. Pada PostgreSQL, data dimasukkan sebagai baris ke dalam tabel, sedangkan pada InfluxDB data dimasukkan sebagai point line protocol sesuai rancangan measurement pada bagian 3.



Untuk operasional query, digunakan program Python yang terhubung ke basis data menggunakan library psycopg dan influxdb-client. Setiap query disimpan di folder queries dan folder writes dengan masing-masing modul berisi versi sql dan versi flux dari query.

6. Metodologi Pemodelan Data

Pada relational database, pemodelan dilakukan secara *entity-driven*, yaitu dengan mengidentifikasi entitas, atribut, dan relasi terlebih dahulu dan menggambarkannya dalam *Entity Relationship Diagram*, termasuk kardinalitas, partisipasi, dan weak entity. ERD kemudian ditransformasikan menjadi relational diagram dan skema tabel yang ternormalisasi. Selanjutnya ditambahkan *primary key*, *foreign key*, dan *check constraint* untuk menjaga integritas data. Aturan bisnis yang tidak dapat dinyatakan sebagai constraint akan dijaga oleh level aplikasi.

Pada time-series database (InfluxDB), pemodelan dilakukan secara *query-driven*, yaitu dirancang berdasarkan pola query yang akan dijalankan. Langkahnya adalah sebagai berikut:

a. Identifikasi data berbasis waktu (kejadian bisnis dan sampel sensor), lalu menjadikan masing-masing sebagai measurement dengan timestamp berupa waktu kejadiannya.
b. Mengelompokkan atribut menjadi tag atau field. Tag dipilih untuk atribut yang sering digunakan untuk filter atau pengelompokan dan memiliki kardinalitas rendah. Field dipilih untuk nilai yang diukur atau diagregasi, termasuk ID.
c. Melakukan denormalisasi dengan menyalin atribut yang dibutuhkan dari tabel master ke setiap point karena InfluxDB tidak mendukung join antar-tabel secara efisien.

7. Analisis Perbandingan Kinerja dengan DBMS Relasional

7.1. Mekanisme Pengujian

Pengujian dilakukan dengan membandingkan PostgreSQL 18 dan InfluxDB 2.7.12 yang dijalankan pada container Docker. Kedua DBMS menggunakan dataset yang sama, yaitu data operasional laundry selama 30 hari yang terdiri atas 2.328 pesanan, 3.003 detail pesanan, 5.634 sesi mesin, dan 138.240 pembacaan sensor.

Pengukuran waktu eksekusi dilakukan dari sisi klien Python secara end-to-end, mencakup waktu eksekusi query, komunikasi jaringan, dan pemrosesan hasil oleh klien. Untuk query read, setiap query dijalankan satu kali sebagai warm-up, kemudian diukur sebanyak lima kali dan dihitung rata-ratanya. Sementara itu, setiap query DML dijalankan satu kali. Setelah pengujian DML selesai, basis data dikembalikan ke kondisi semula agar setiap pengujian menggunakan kondisi awal yang setara

7.2. Hasil Eksekusi Query Read

Query	Deskripsi	PostgreSQL	InfluxDB
R1	Jumlah pesanan per hari	2.17 ms	6.62 ms
R2	Jumlah total pembayaran per hari	2.66 ms	9.53 ms
R3	Jumlah total berat cucian per hari	5.46 ms	9.24 ms
R4	Pendapatan per jenis layanan laundry (hanya pesanan yang sudah dibayar)	3.27 ms	409.73 ms
R5	Jumlah pesanan berdasarkan status	1.56 ms	9.83 ms
R6	Rata-rata suhu dan daya per mesin per jam	88.99 ms	238.33 ms
R7	Total energi (kWh) per jenis layanan laundry	313.14 ms	1288.95 ms

7.3. Analisis Hasil Query Read

PostgreSQL memiliki waktu eksekusi lebih cepat pada seluruh query read yang diuji. Pada agregasi sederhana (R1-R3, R5), selisih waktu eksekusinya kecil karena data relatif sedikit dan dapat dipindai langsung di memori. Sementara itu, InfluxDB menanggung overhead tambahan berupa parsing query Flux, komunikasi HTTP, dan pengiriman hasil dalam format annotated CSV yang harus diproses ulang oleh klien.

Perbedaan eksekusi yang paling besar terlihat pada R4 dan R7 karena kedua query memerlukan penggabungan data dari beberapa sumber. Pada R4, detail pesanan perlu dicocokkan dengan data pembayaran berdasarkan order_id. Sementara, order_id disimpan sebagai field, sehingga query Flux perlu melakukan pivot dan join di memori. Akibatnya, waktu eksekusi InfluxDB mencapai 125,3x lipat lebih lama daripada PostgreSQL. Pada R7, setiap pembacaan sensor harus dikaitkan dengan sesi mesin berdasarkan rentang waktu. PostgreSQL menanganinya dengan range join dan lead(), sedangkan Flux memerlukan union, pengurutan, fill(usePrevious), dan integral.

Pada R6, PostgreSQL juga lebih cepat dibandingkan InfluxDB. Hasil ini berbeda dari perkiraan awal bahwa InfluxDB akan lebih unggul pada agregasi data time-series. Salah satu kemungkinan penyebabnya adalah ukuran dataset yang masih relatif kecil sehingga seluruh data sensor dapat dimuat di memori PostgreSQL. Selain itu, indeks pada (machine_id, recorded_at) PostgreSQL serta kebutuhan pivot dan pengiriman 11.520 baris hasil di InfluxDB juga menjadi kemungkinan penyebab hasil tersebut.

Secara keseluruhan, hasil pengujian menunjukkan bahwa PostgreSQL lebih efisien pada dataset dan pola query yang digunakan. Namun, pengujian ini belum dapat menunjukkan kinerja kedua DBMS ketika jumlah data sensor meningkat hingga jutaan point. Oleh karena itu, hasil tersebut belum cukup untuk menyimpulkan bahwa PostgreSQL selalu lebih unggul pada seluruh skenario pengolahan data time-series.

7.4. Hasil Eksekusi Query DML

Query	Deskripsi	PostgreSQL	InfluxDB
D1	Menambahkan pesanan beserta detailnya	7.47 ms	4.90 ms
D2	Mencatat pembayaran dan memperbarui status pesanan	4.81 ms	13.59 ms
D3	Menambahkan 192 pembacaan sensor	18.74 ms	6.24 ms
D4	Mengoreksi satu pembacaan sensor	4.60 ms	6.14 ms
D5	Menghapus data sensor yang lebih lama dari tanggal tertentu	12.24 ms	11.90 ms

7.5. Analisis Query DML

Hasil pengujian DML menunjukkan bahwa keunggulan masing-masing DBMS bergantung pada jenis operasi dan cara identitas data direpresentasikan. Pada operasi penambahan data, InfluxDB lebih cepat (terlihat pada D1 dan D3). Hal ini karena InfluxDB dapat menerima data dalam satu batch menggunakan line protocol, sedangkan PostgreSQL perlu menjalankan transaksi, memeriksa constraint, dan memperbarui indeks. Hasil ini menunjukkan potensi InfluxDB untuk menangani ingestion data sensor dalam jumlah besar dan berkelanjutan.

Sebaliknya, PostgreSQL lebih efisien pada operasi yang melibatkan kunci bisnis. Pada D2, pencatatan pembayaran dan pembaruan status dilakukan dengan INSERT dan UPDATE berdasarkan order_id dalam satu transaksi. Di InfluxDB, klien harus mencari point pesanan terlebih dahulu karena order_id berupa field, lalu menulis pembayaran, status, dan

pengiriman secara terpisah. Operasi tersebut tidak atomik sehingga konsistensinya harus dijaga oleh aplikasi.

Pada D4 dan D5, waktu eksekusi kedua DBMS relatif setara, tetapi mekanismenya berbeda. Koreksi sensor (D4) cukup efisien di InfluxDB karena identitas point sudah diketahui dari mesin dan waktu, sehingga point dapat langsung ditimpa. Untuk D5, InfluxDB hanya mendukung penghapusan berdasarkan rentang waktu serta predikat measurement atau tag, bukan nilai field, sehingga penghapusan berdasarkan order_id tidak dapat dilakukan langsung seperti di PostgreSQL.

7.6. Analisis Perbandingan dari Sisi Pemodelan Data

Aspek	PostgreSQL (Relational)	InfluxDB (TimeSeries)
Pendekatan	Entity-driven, ternormalisasi	Query-driven, terdenormalisasi
Skema	Didefinisikan di awal (schema-on-write) dengan tipe data ketat	Schemaless: tipe field ditentukan oleh penulisan pertama
Identitas data	Primary key	Measurement, tag set, timestamp
Relasi	Foreign key dan join	Tidak ada join atas field atau rentang waktu di Flux
Integritas	PK, FK, CHECK, UNIQUE, transaksi ACID	Tidak ada constraint maupun transaksi
Data master	Tabel tersendiri	hanya atribut yang dibutuhkan yang disalin ke point

Berdasarkan itu, model relasional lebih fleksibel terhadap kebutuhan query baru, sedangkan model InfluxDB harus dirancang sejak awal dari query yang dijalankan. Keputusan tag vs field juga sangat berpengaruh terhadap kinerja hasil.

7.7. Kelebihan dan Kekurangan PostgreSQL dan InfluxDB

Berdasarkan analisis perbandingan kinerja query dan pemodelan data serta informasi dari dokumentasi kedua database, dapat disimpulkan kelebihan dan kekurangan sebagai berikut:

PostgreSQL (Relational)	InfluxDB (TimeSeries)
Kelebihan	
1. Mendukung relasi antartabel melalui JOIN. 2. Integritas data terjaga oleh primary key, foreign key, CHECK constraint, dan transaksi.	1. Menyediakan model time-series yang secara langsung merepresentasikan sensor & kejadian berbasis timestamp. 2. Mendukung penulisan data secara batch melalui line protocol.

3. Mendukung pembaruan beberapa data terkait dalam satu transaksi.	3. Point yang diketahui identitasnya dapat langsung ditimpa/ditulis ulang
Kekurangan	
1. Penambahan data dapat membutuhkan waktu lebih lama karena transaksi, pemeriksaan constraint, dan pemeliharaan indeks. 2. Analisis berbasis waktu yang melibatkan banyak data dapat memerlukan query dengan agregasi, window function, atau penggabungan tabel yang kompleks.	1. Tidak ada foreign key, check constraint, dan transaksi sehingga konsistensi data dijaga level aplikasi 2. Penggabungan data berdasarkan field membutuhkan pemrosesan tambahan. 3. Perubahan point dapat menimpa data dengan kombinasi measurement, tag, dan timestamp yang sama. 4. Penghapusan data berdasarkan field tidak dapat dilakukan secara langsung
Kasus Penggunaan Terbaik	
Sistem transaksi yang membutuhkan relasi antardata, validasi integritas, pembaruan beberapa entitas secara konsisten, dan query yang melibatkan banyak tabel.	Penyimpanan dan analisis pembacaan sensor atau kejadian berbasis waktu, terutama ketika data baru masuk secara berkala atau dalam batch dan query banyak menggunakan rentang waktu serta agregasi.

8. Analisis Perbandingan Kinerja dengan Document Database (Bonus)

Pengujian ini dilakukan dengan eksekusi yang terpisah dari bagian sebelumnya

Query	Deskripsi	PostgreSQL	InfluxDB	MongoDB
R1	Jumlah pesanan per hari	3.35 ms	9.27 ms	5.75 ms
R2	Jumlah total pembayaran per hari	2.74 ms	9.16 ms	6.16 ms
R3	Jumlah total berat cucian per hari	4.95 ms	8.67 ms	8.95 ms
R4	Pendapatan per jenis layanan laundry (hanya pesanan yang sudah dibayar)	3.68 ms	483.42 ms	6.09 ms
R5	Jumlah pesanan berdasarkan status	1.88 ms	11.81 ms	3.31 ms
R6	Rata-rata suhu dan daya per mesin per jam	111.24 ms	270.42 ms	125.77 ms
R7	Total energi (kWh) per jenis layanan laundry	299.45 ms	1434.56 ms	1959.13 ms
D1	Menambahkan pesanan beserta detailnya	10.70 ms	6.82 ms	1.83 ms

D2	Mencatat pembayaran dan memperbarui status pesanan	4.88 ms	10.73 ms	2.19 ms
D3	Menambahkan 192 pembacaan sensor	22.13 ms	7.63 ms	7.62 ms
D4	Mengoreksi satu pembacaan sensor	4.42 ms	4.84 ms	7.84 ms
D5	Menghapus data sensor yang lebih lama dari tanggal tertentu	19.15 ms	17.88 ms	18.64 ms

Berdasarkan hasil pengujian, terlihat bahwa PostgreSQL tercepat pada seluruh query read, MongoDB menempati posisi kedua pada hampir semua query read (R1, R2, R4, R5, R6) dan jauh lebih cepat daripada InfluxDB pada R4 (6,09 ms dibandingkan 483,42 ms). Hal ini kemungkinan karena data pesanan, detail, dan pembayaran disimpan dalam satu dokumen sehingga pendapatan per layanan tidak memerlukan join, sedangkan InfluxDB harus melakukan pivot dan join karena order_id berupa field. Pada R7, terlihat bahwa MongoDB merupakan yang paling lambat (1.959,13 ms). Hal ini mungkin karena mengaitkan pembacaan sensor dengan sesi mesin berdasarkan rentang waktu lebih kompleks pada model dokumen.

Pada operasi tulis, MongoDB unggul pada data transaksi D1 dan D2, jauh lebih cepat daripada PostgreSQL dan InfluxDB. Hal ini karena satu pesanan dapat ditulis atau diperbarui sebagai satu dokumen tanpa join atau penulisan terpisah. Pada penambahan data sensor (D3), InfluxDB dan MongoDB setara (7,63 dan 7,62 ms) dan sekitar tiga kali lebih cepat daripada PostgreSQL (22,13 ms). Artinya, MongoDB juga memiliki keunggulan dalam menambahkan data berjumlah besar seperti InfluxDB yang memiliki keunggulan batch ingest. Terakhir, pada D5 terlihat bahwa ketiga database cukup setara.

Secara keseluruhan, tidak ada DBMS yang unggul pada semua pola akses. PostgreSQL paling konsisten untuk query baca dan query lintas entitas, MongoDB cocok untuk akses per pesanan tetapi lemah pada analisis lintas koleksi berbasis waktu, dan InfluxDB hanya menonjol pada ingest sensor secara batch.

9. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian, setiap DBMS memiliki keunggulannya masing-masing dan pemodelan data perlu disesuaikan dengan pola akses yang dominan. Relational database relatif fleksibel untuk berbagai kebutuhan query, termasuk query yang melibatkan relasi antar data. Time-series database perlu dirancang berdasarkan kebutuhan query sejak awal dan kurang sesuai untuk penggabungan data berdasarkan ID, tetapi cocok untuk penyimpanan data sensor dan kejadian berbasis waktu yang masuk secara batch. Sementara itu, document database unggul dalam penulisan data pesanan yang disimpan dalam satu dokumen, tetapi memiliki keterbatasan pada analisis lintas koleksi berbasis waktu. Kesimpulan ini terbatas pada dataset dan skenario pengujian yang digunakan, sehingga kinerja time-series database pada skala besar belum dapat dipastikan.

10. Lampiran

Repository: https://github.com/naylzhra/Project2_13523079